

Introduction to Embedded Systems

Course Code: 244066

Chapter 4: Cortex-M3 Exceptions and STM32 Interrupts

By

Dr. Muhammad Imran Nadeem

Zhengzhou University

Contents

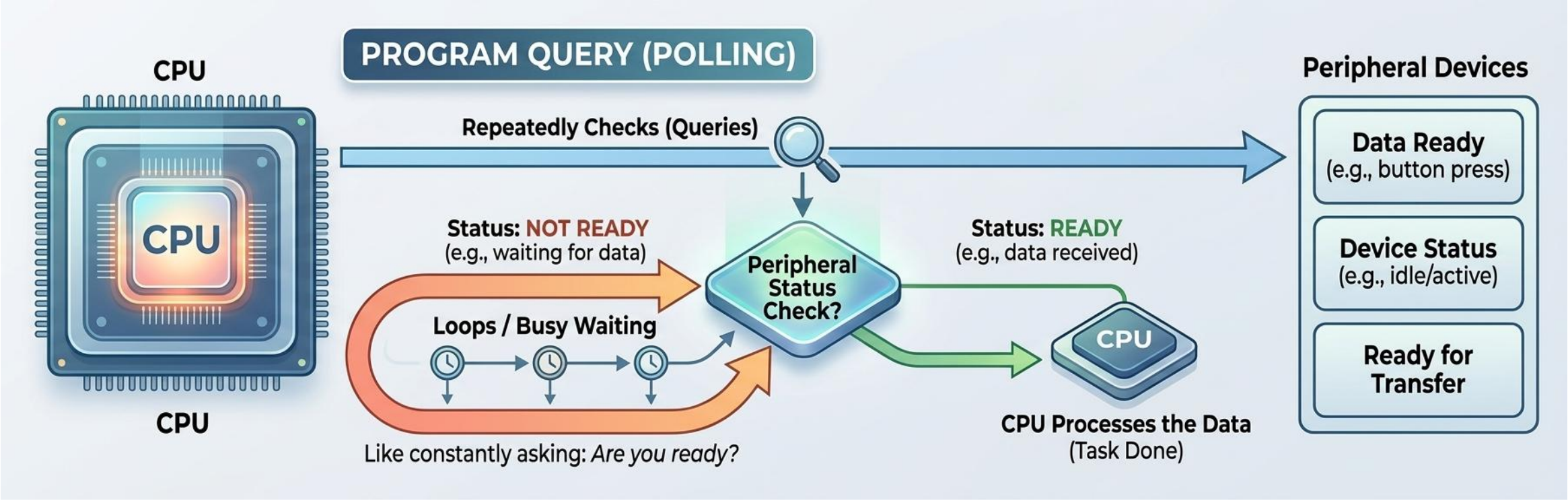
1. **Program Query (Polling)**
2. Interrupt-Driven Mode
3. Exception
4. Cortex-M3 Exception Model
5. STM32 Interrupt Architecture
   - NVIC
   - NVIC Priority
6. External Interrupt (EXTI)
7. Summary

4.1. Program Query (Polling)

Program Query (Polling)

- CPU repeatedly checks **(queries)** a peripheral's status register.
- If the status indicates "**ready**" (e.g., data received, button pressed), the CPU processes it.
- If **not ready**, CPU keeps looping (busy waiting).

4.1. Program Query (Polling)



4.1. Program Query (Polling)

- **Pros of Polling**

Simple to implement.

Good for very small/simple systems.

Easy to debug and understand.

- **Cons of Polling**

Wastes CPU time (busy waiting).

Not efficient for real-time systems.

Many devices need checking.

Main loop becomes very slow.

CPU may miss events if polling is too slow.

4.1. Program Query (Polling)

Analogy: Imagine waiting for a bus.

- ***Polling*** : you keep stepping outside every 5 seconds to check.



- ***Interrupt***: The bus honks when it arrives (saves your energy).



Contents

1. Program Query (Polling)
2. **Interrupt-Driven Mode**
3. Exception
4. Cortex-M3 Exception Model
5. STM32 Interrupt Architecture
   - NVIC
   - NVIC Priority
6. External Interrupt (EXTI)
7. Summary

4.2. Interrupt-Driven Mode

Interrupt-Driven Mode

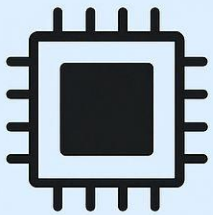
An **interrupt** is a signal that tells the CPU to *temporarily stop* its execution and **handle an event** immediately.

- The CPU **does not waste time checking** device status (like in polling).
- Instead, the **device** itself "raises its hand" and says: *"I need attention now!"*
- When the event is handled, the CPU **resumes** to where it left off.

4.2. Interrupt-Driven Mode

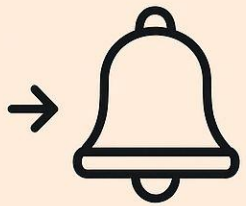
Polling vs Interrupt-Driven Approaches

POLLING



- CPU continuously checks status
- Potentially wastes CPU time

INTERRUPT-DRIVEN



- CPU responds only when needed
- More efficient use of CPU

4.2. Interrupt-Driven Mode

Analogy

- **Polling**: A teacher keeps asking every student "Do you have a question?"
- **Interrupt**: A student raises their hand only when they have a question.



4.2. Interrupt-Driven Mode

Benefits in Embedded Systems

- **Efficient CPU Usage:** CPU can perform other tasks or stay in low-power mode until an event happens.
- **Real-Time Response:** Immediate reaction to external events (button press, sensor signal, incoming data).
- **Scalability:** Easier to handle multiple devices without constant checking.
- **Power Saving:** Reduces energy waste, important for battery-powered systems.
- **Better System Performance:** Frees CPU from busy waiting → smoother multitasking.

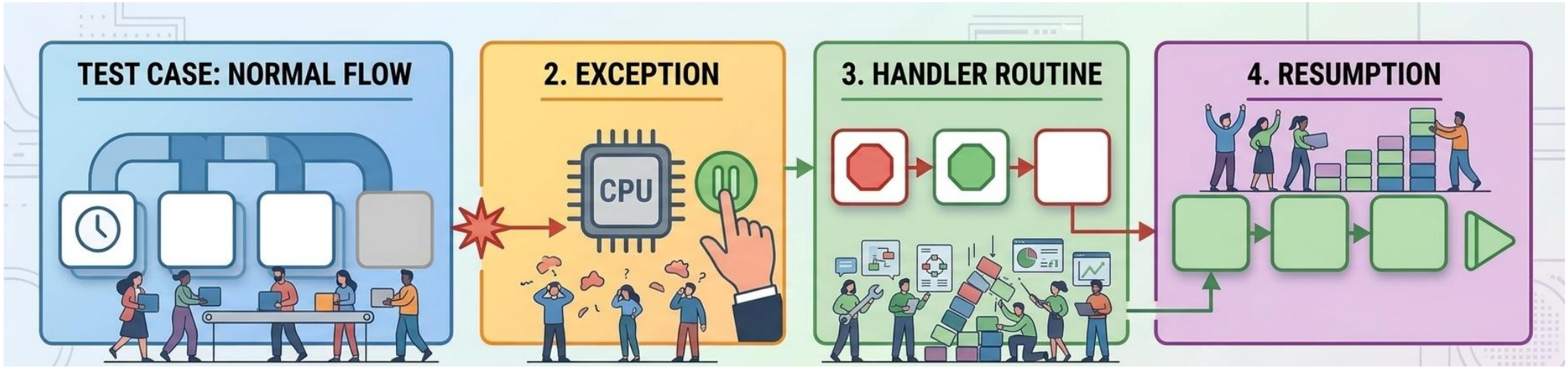
Contents

1. Program Query (Polling)
2. Interrupt-Driven Mode
3. **Exception**
4. Cortex-M3 Exception Model
5. STM32 Interrupt Architecture
   - NVIC
   - NVIC Priority
6. External Interrupt (EXTI)
7. Summary

4.3. Exception

What is an Exception?

- An **exception** is an *event* that temporarily changes the normal flow of a program.
- It tells the **CPU**: *"Something special happened, stop current execution and handle me first."*
- After handling the event, the CPU resumes where it left off.

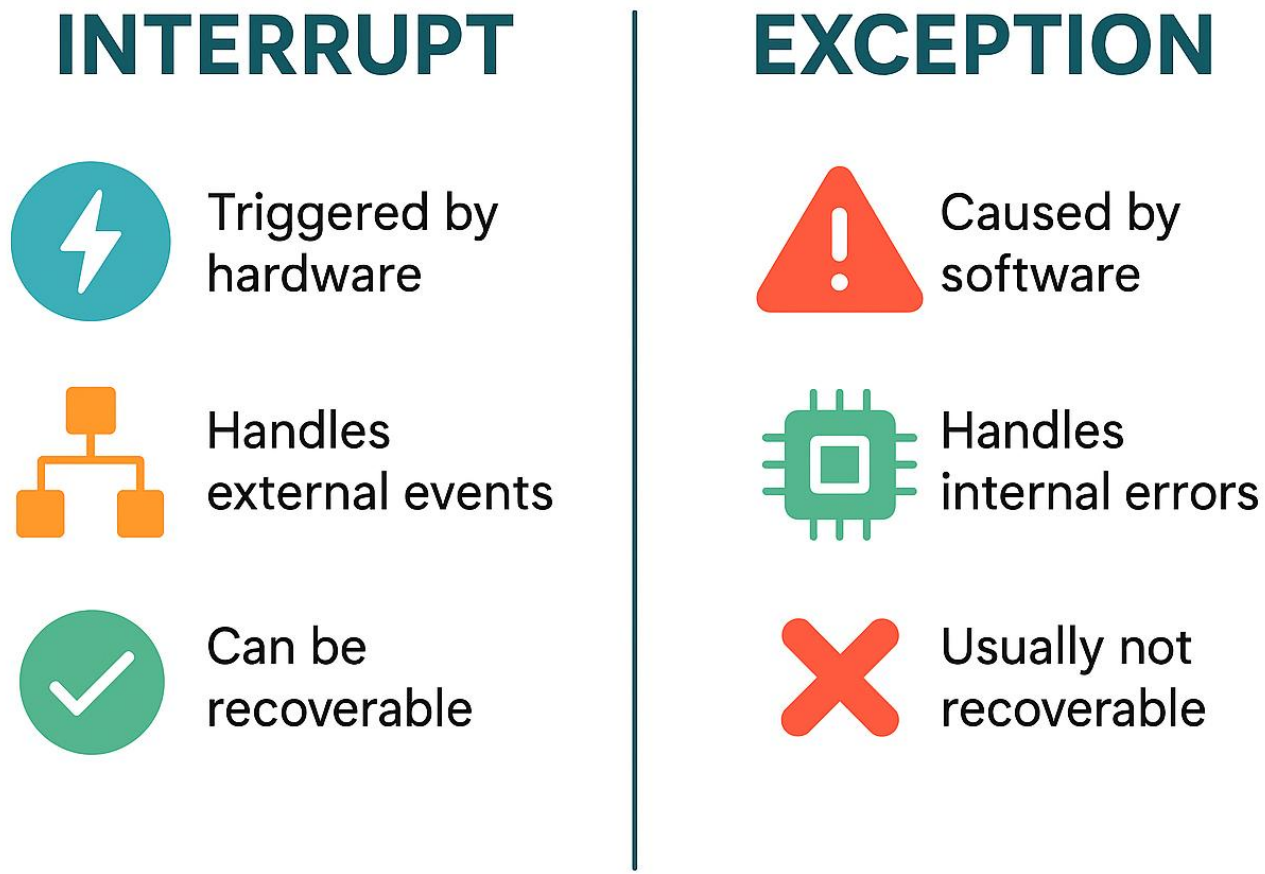


4.3. Exception

- **Analogy**
  - Imagine you are studying (main program). Suddenly, your phone rings (exception).
  - You pause studying, answer the call (ISR), then return to study after the call ends.



4.3. Exception



4.3. Exception

How Exceptions Work

1. An event occurs (reset, fault, or peripheral request).
2. CPU **saves its current state** (registers, program counter).
3. CPU jumps to a predefined **Exception Handler (ISR)**.
4. Handler executes instructions to solve the event.
5. CPU **restores state** and continues normal program flow.

4.3. Exception

Why Exceptions are Important?

- Enable **real-time response** to critical events.
- Improve **system reliability** (fault detection).
- Support **multitasking** in embedded systems.

4.3. Exception

Types of Exceptions (in Cortex-M3 / STM32)

1. Reset Exception
2. NMI (Non-Maskable Interrupt)
3. Fault Exceptions
    i. Hard Fault: severe system error.
    ii. Memory Fault: invalid memory access.
    iii. Bus Fault: data transfer error.
    iv. Usage Fault: invalid instruction or operation.
4. Interrupt Requests (IRQs)
5. Generated by peripherals (GPIO, USART, Timer, ADC, etc.).

4.3. Exception

1. Reset Exception

- A **Reset Exception** happens when:
  - The system is **powered on** (cold start).
  - The **reset button** is pressed (manual restart).
- It is the very **first event** that occurs in any microcontroller system.

4.3. Exception

How it Works

- **CPU starts from a fixed memory address**:
  - This address points to the **vector table**.
  - The first entry is the **initial stack pointer**.
  - The second entry is the **reset handler address**.
- The CPU jumps to the **reset handler** → which runs startup code.
- Startup code initializes memory, peripherals, and then calls the main() function.

4.3. Exception

Why it is Important

- Ensures the CPU always has a **known starting point**.
- Brings the system into a **clean, predictable state**.
- Prevents undefined behavior after power-up.

Analogy

- Think of a **reset exception** like **restarting your computer**:
  - **Power on** → BIOS/UEFI starts → Loads OS → Runs your programs.
  - In microcontrollers: Power on → Reset Handler → Startup code → main().



4.3. Exception

2. NMI (Non-Maskable Interrupt)

- **NMI** stands for **Non-Maskable Interrupt**.
- It is a special kind of interrupt with the **highest priority**.
- Unlike normal interrupts, it **cannot be disabled (masked)** by software.



4.3. Exception

Key Characteristics

- **Highest Priority**
  - Always serviced first, before any other interrupt.
- **Non-Maskable**
  - Cannot be ignored or disabled by the programmer.
  - Ensures urgent events are always handled.
- **Special Use**
  - Reserved for **critical errors** or **emergency situations**.

4.3. Exception

Typical Use Cases

- **Hardware failure** detection (e.g., memory errors).
- **Power failure warning** (system can quickly save state).
- **External emergency signal** (e.g., watchdog timer alert).



4.3. Exception

How it Works

- Event triggers NMI line.
- CPU **immediately suspends** current execution.
- Jumps to **NMI Handler** (predefined in vector table).
- Executes urgent code (error handling, safe shutdown).
- Returns to the main program (if possible).

4.3. Exception

- **Analogy :** Imagine you're in class:

- **Regular interrupts** = students raising hands (teacher decides priority).



- **NMI = fire alarm,** everyone must stop and evacuate, no exceptions.



4.3. Exception

3. Fault Exceptions

- Exceptions are not only for external events (like button presses), but also for **errors inside the system**.
- They help the CPU detect and respond to problems.
  - **Hard Fault**: severe system error.
  - **Memory Fault**: invalid memory access.
  - **Bus Fault**: data transfer error.
  - **Usage Fault**: invalid instruction or operation.

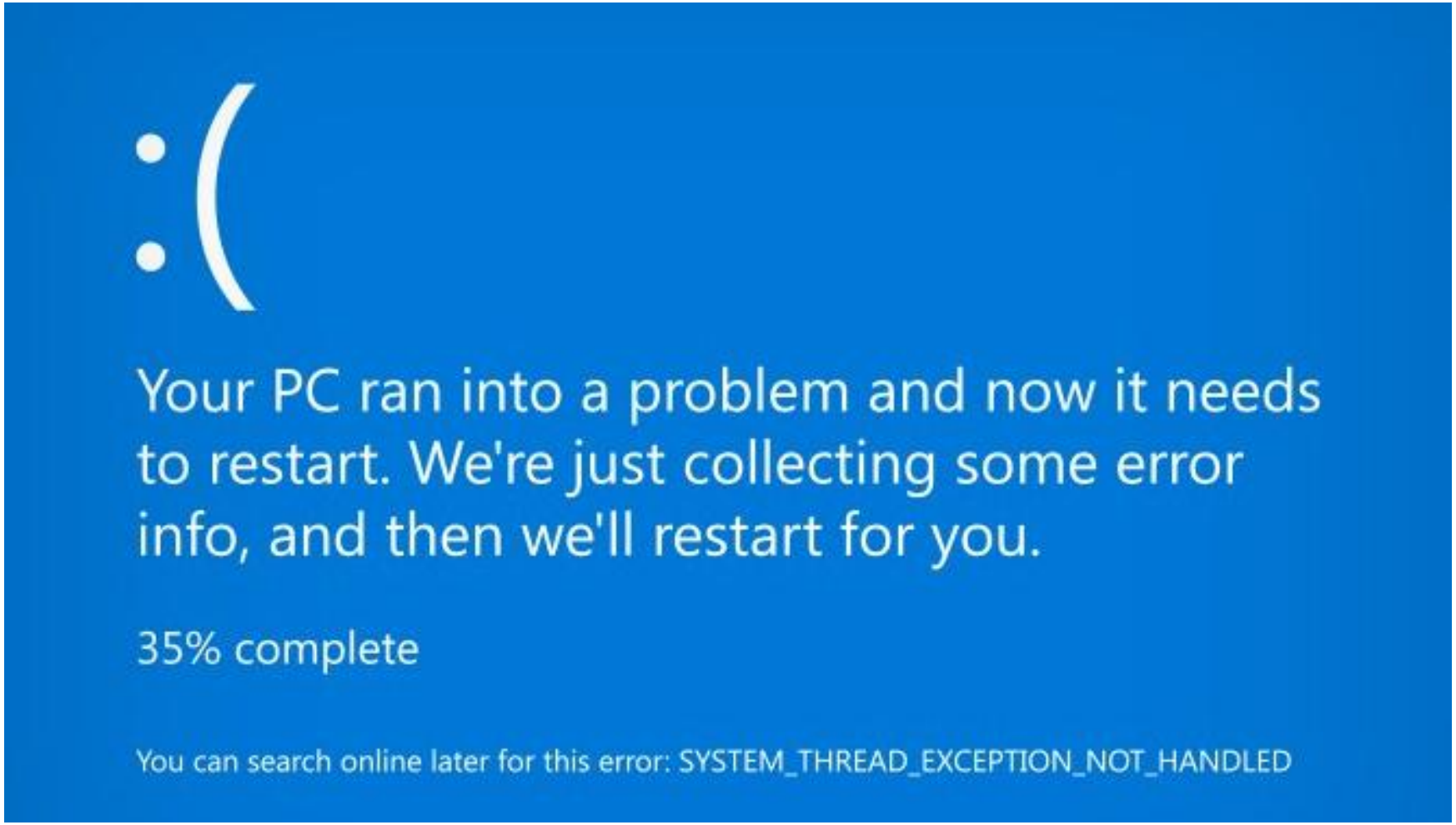
4.3. Exception

i. Hard Fault

- Severe system error when no other handler can deal with the problem.
- **Examples:**
  - Illegal memory access.
  - Executing an undefined instruction.
- **Effect:** Always triggered if another fault cannot be handled.

4.3. Exception

- **Analogy:** Like a "system crash" → everything stops until you fix it.



4.3. Exception

ii. Memory Fault

- Triggered when the program tries to access invalid or protected memory.
- **Examples**:
  - Reading from a non-existent address.
  - Writing into read-only memory.

4.3. Exception

- **Analogy**: Trying to open a locked room with the wrong key.



4.3. Exception

iii. Bus Fault

- Error in data transfer between CPU and memory/peripherals.
- **Examples:**
  - Invalid alignment (e.g., trying to read 4 bytes from an odd address).
  - Peripherals not responding during transfer.

Analogy: Like sending a package but the delivery truck breaks down → data never arrives.



4.3. Exception

iv. Usage Fault

- Error caused by executing an **invalid instruction or operation**.
- **Examples:**
  - Division by zero.
  - Using an instruction not supported by CPU.
- **Analogy:** Asking someone to do a task they don't know how to do.



4.3. Exception

Why Fault Exceptions are Important?

- They **prevent silent errors** in the system.
- Help engineers **debug programs** (via fault handlers).
- Ensure the system **fails safely** instead of running with corrupt data.

4.3. Exception

Fault exceptions act like **safety nets** that catch mistakes and alert the system,

instead of letting it continue incorrectly.



4.3. Exception

4. Interrupt Requests (IRQs)

- An **IRQ (Interrupt Request)** is a signal generated by a **peripheral device** to request the CPU's attention.
- Instead of the CPU constantly checking (polling), the device itself *notifies* the CPU when it has data or needs service.

4.3. Exception

5. Generated by peripherals (STM32)

- **GPIO:** Button press → triggers an external interrupt.
- **USART (Serial Communication):** New byte of data received → triggers an interrupt.
- **Timer:** Timer overflow → triggers an interrupt for periodic tasks.
- **ADC (Analog-to-Digital Converter):** Conversion complete → triggers an interrupt to process the value.

4.3. Exception

How IRQs Work

1. Peripheral generates IRQ.
2. CPU is interrupted → saves its current state.
3. CPU jumps to the **Interrupt Service Routine (ISR)**.
4. ISR handles the event (e.g., read data, clear flag).
5. CPU resumes the main program.

4.3. Exception

Benefits of IRQs

- Efficient: CPU only responds when needed.
- Supports **real-time applications**.
- Allows **parallel activity** (CPU and peripherals working together).
- Essential for handling **multiple devices** in embedded systems.

Contents

1. Program Query (Polling)
2. Interrupt-Driven Mode
3. Exception
4. **Cortex-M3 Exception Model**
5. STM32 Interrupt Architecture
   - NVIC
   - NVIC Priority
6. External Interrupt (EXTI)
7. Summary

4.4. Cortex-M3 Exception Model

Cortex-M3 Exception Model

The Cortex-M3 microcontroller has a **well-defined exception handling model** that makes interrupt management efficient and reliable.

1. **Vector Table**: List of exception/ISR addresses.
2. **Stacking**: CPU saves registers automatically.
3. **Nested execution**: Higher priority interrupts can preempt lower ones

4.4. Cortex-M3 Exception Model

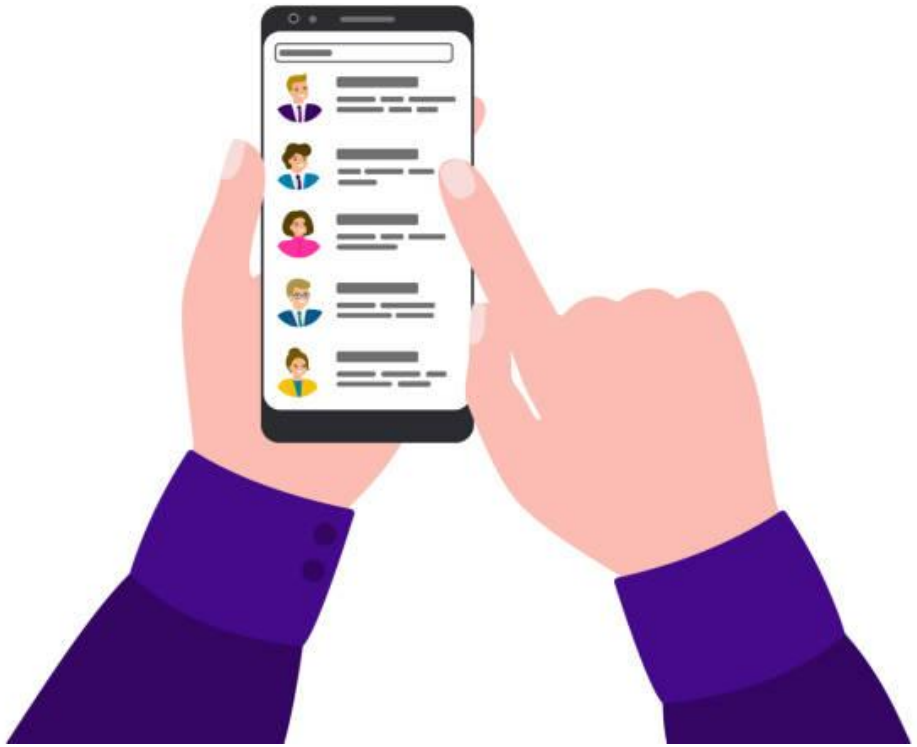
1. Vector Table

- A **vector table** is a lookup table stored in memory.
- Each entry in the table holds the **address of an Exception Handler (ISR)**.
- When an event (reset, fault, or IRQ) occurs, the CPU looks up the corresponding address in the vector table and jumps to it.

4.4. Cortex-M3 Exception Model

- **Example:**
  - Reset → Jumps to Reset Handler.
  - EXTI (GPIO interrupt) → Jumps to EXTI Handler.

- **Analogy:** Like a **phone contact list**: instead of remembering numbers, you look up the correct person to call.



4.4. Cortex-M3 Exception Model

2. Stacking (Automatic Register Saving)

- When an exception occurs:
  - Cortex-M3 **automatically saves** important CPU registers (Program Counter, xPSR, etc.) onto the stack.
  - This ensures the CPU can **resume the exact program state** after the handler finishes.
- After ISR execution, these registers are **automatically restored**.

- **Benefit:** Programmer does not need to manually save/restore context → faster and safer.

4.4. Cortex-M3 Exception Model

- **Analogy:** Like putting a **bookmark** in your book before taking a break → you can resume exactly where you left off.



4.4. Cortex-M3 Exception Model

3. Nested Execution (Interrupt Priorities)

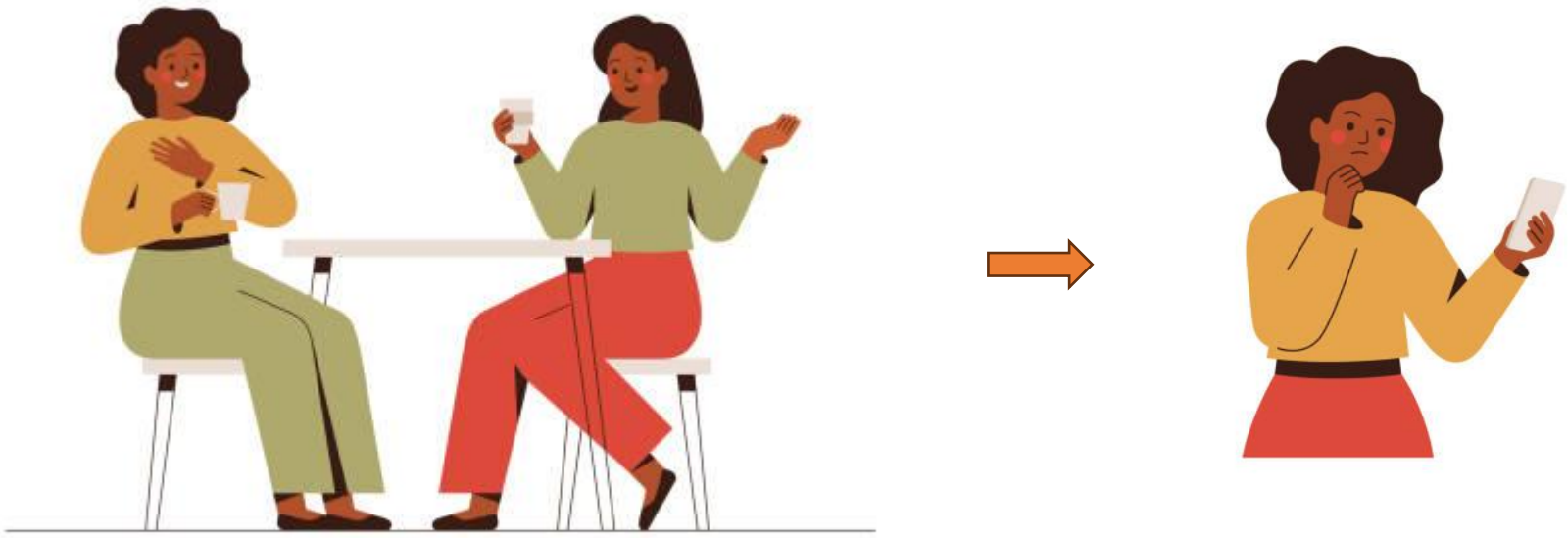
- Cortex-M3 allows **nested interrupts**.
- A **higher-priority interrupt** can preempt (interrupt) a lower-priority one.
- Ensures urgent events are serviced quickly.

4.4. Cortex-M3 Exception Model

- **Example:**
  - USART RX interrupt (low priority) is running.
  - Timer overflow interrupt (higher priority) occurs → CPU suspends USART ISR, runs Timer ISR, then returns to USART ISR.

4.4. Cortex-M3 Exception Model

- **Analogy:** If you're talking to a friend and your **phone rings with an emergency call**, you pause the chat and answer the urgent call first.



4.4. Cortex-M3 Exception Model

Why Cortex-M3 Exception Model is Powerful

- **Fast & efficient** handling of exceptions.
- **Automatic context saving** reduces programmer workload.
- **Nested execution** supports real-time response.

Contents

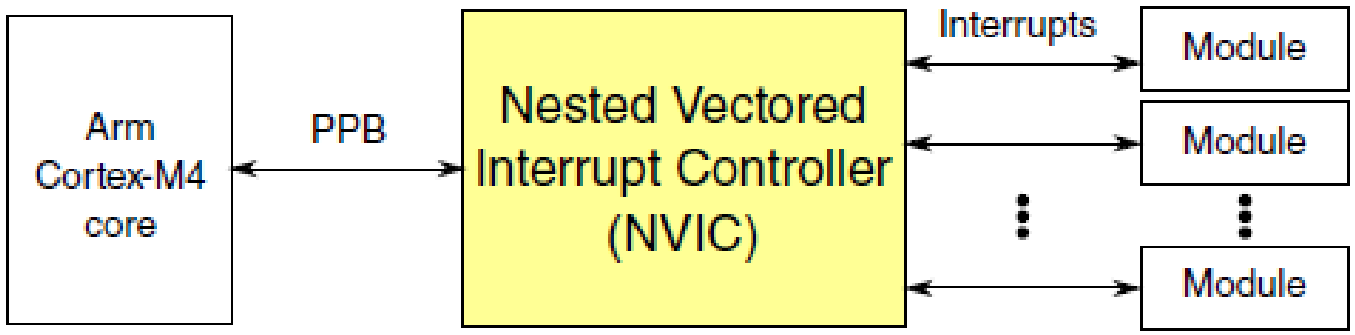
1. Program Query (Polling)
2. Interrupt-Driven Mode
3. Exception
4. Cortex-M3 Exception Model
5. **STM32 Interrupt Architecture**
   - NVIC
   - NVIC Priority
6. External Interrupt (EXTI)
7. Summary

4.5. STM32 Interrupt Architecture

STM32 Interrupt Architecture

Nested Vectored Interrupt Controller (NVIC).

- NVIC is a special hardware unit inside **ARM Cortex-M** microcontrollers.
- It **manages** all the **interrupts** coming from peripherals and exceptions.
- Acts like a **traffic controller**: decides which interrupt the CPU should handle first.



4.5. STM32 Interrupt Architecture

Key Features

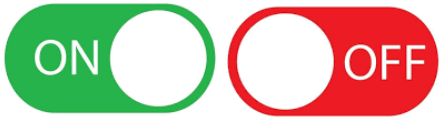
1. Priority Levels

- NVIC assigns each interrupt a **priority number**.
- **Higher** priority interrupts can **preempt** (interrupt) lower priority ones.
- Supports ***nested*** *interrupts* → an interrupt can happen inside another.

Example: Timer interrupt (**high priority**) can interrupt a UART receive (**low priority**).

4.5. STM32 Interrupt Architecture

Key Features



2. Enable/Disable Specific IRQs

- Each interrupt source can be turned ON or OFF.
- Useful when some peripherals are not in use.
- **Example**: Disable USART interrupts when no serial data is expected.

4.5. STM32 Interrupt Architecture

Key Features

3. Automatic Context Saving/Restoring

- When an interrupt occurs:
  - NVIC + CPU **automatically save registers** (so work is not lost).
  - After ISR completes, CPU **restores registers** and continues main program.
- Makes interrupt handling **fast and reliable**.

4.5. STM32 Interrupt Architecture

Why NVIC is Important?

- Provides **real-time responsiveness**.
- Allows **multiple devices** to generate interrupts without confusion.
- Reduces programmer burden (hardware handles saving/restoring).
- Supports **nested, prioritized execution** (critical for embedded systems).



4.5. STM32 Interrupt Architecture

- **Analogy:** Imagine a **hospital emergency room**:
  - Patients (interrupts) come in with different severity levels (priorities).
  - The triage nurse (NVIC) decides who gets treated first.
  - Doctors (CPU) pause current treatment if a more urgent patient arrives.





4.5. STM32 Interrupt Architecture

NVIC Priority

The **Nested Vectored Interrupt Controller (NVIC)** allows interrupts to be organized by **priority levels**, so the CPU knows which event is more **important** when multiple interrupts occur.



4.5. STM32 Interrupt Architecture

1. Preemption Priority

- Determines whether **one interrupt can interrupt another**.
- A higher preemption priority interrupt can stop (preempt) a lower one in progress.
- **Example**:
  - **Timer IRQ (priority 1)** can interrupt **USART IRQ (priority 2)**.
  - After Timer ISR finishes, CPU resumes USART ISR.

4.5. STM32 Interrupt Architecture

- **Analogy:** Like an emergency exit if the fire alarm goes off → emergency first!





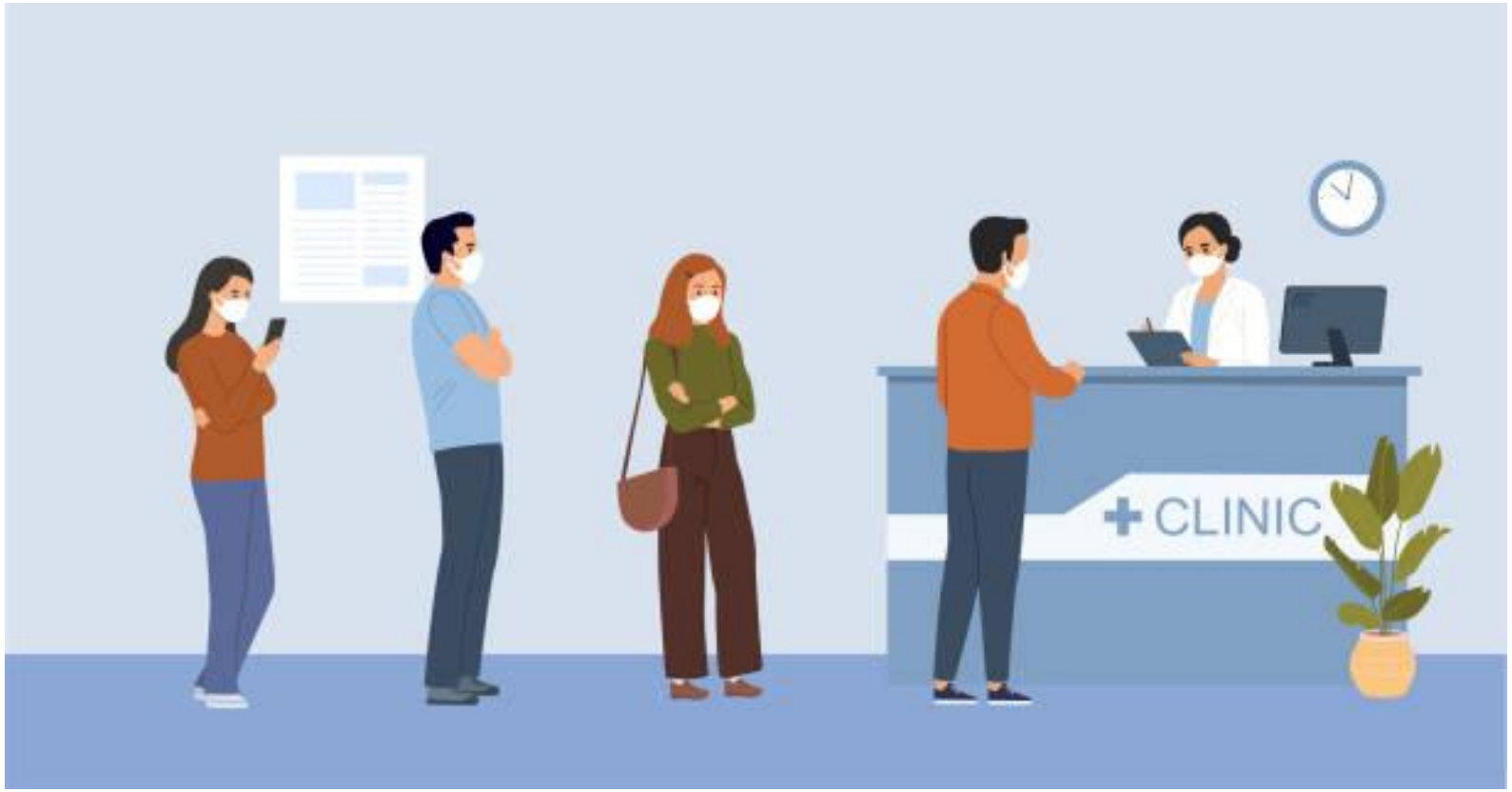
4.5. STM32 Interrupt Architecture

2. Subpriority

- Used when **two interrupts have the same preemption priority**.
- Decides **which interrupt is handled first** if both are pending at the same time.
- **Example**:
  - Two interrupts with priority level 2 → USART IRQ has higher subpriority than EXTI IRQ → USART runs first.

4.5. STM32 Interrupt Architecture

- **Analogy:** Two patients with the same medical urgency arrive at a hospital. The nurse (NVIC) decides who gets treated first based on arrival order (subpriority).



4.5. STM32 Interrupt Architecture

PREEMPTION



Higher-priority interrupts preempt lower-priority ones

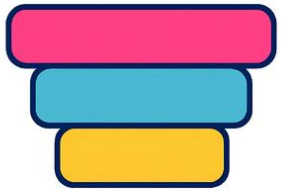
priority

Higher-priority interrupts preempt lower-priority ones

SUBPRIORITY



Order if priorities are equal



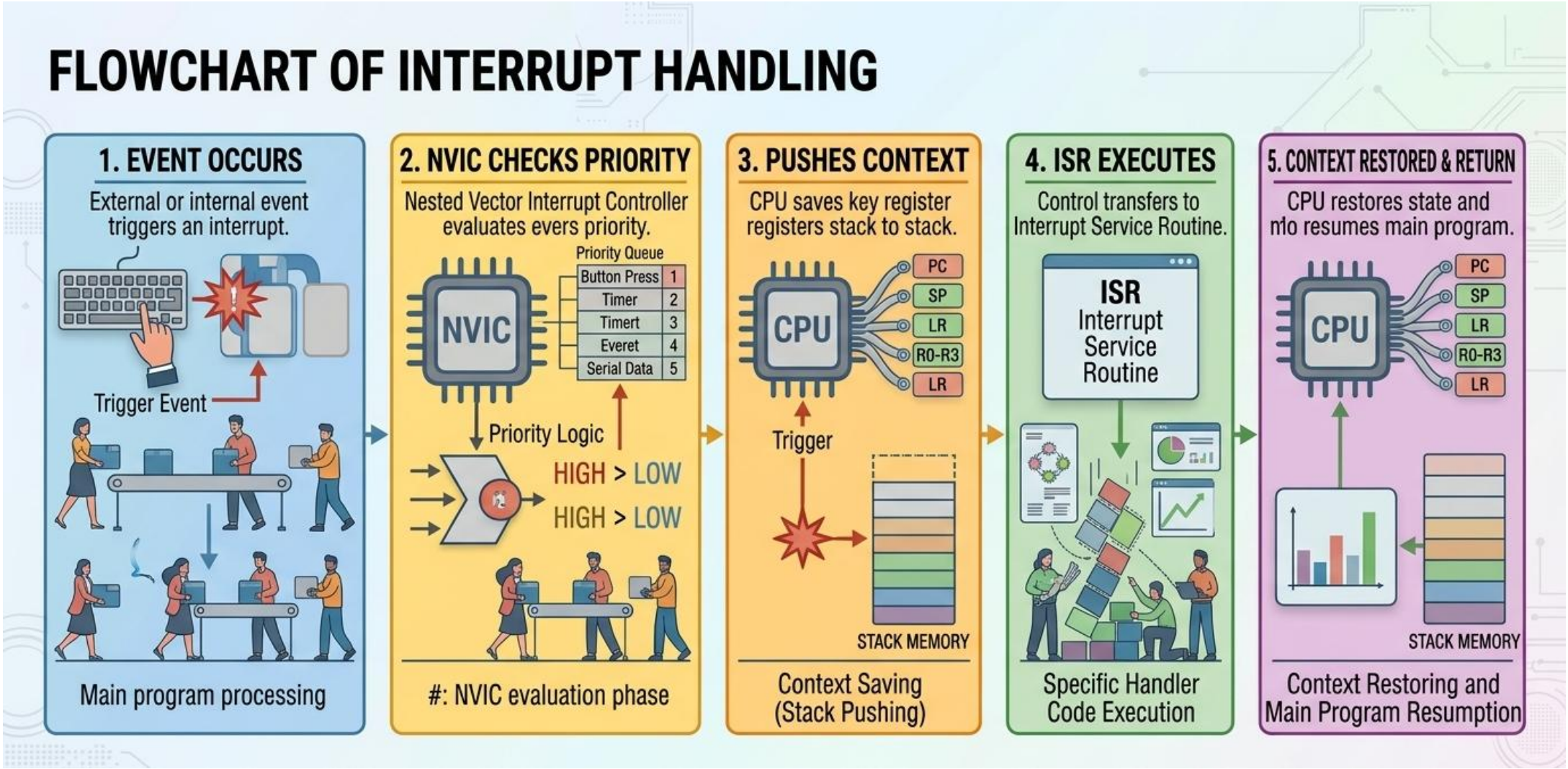
Sorts interrupts of the same preemption

4.5. STM32 Interrupt Architecture

Why Priority is Important?

- Ensures **critical tasks** (like safety monitoring) always get immediate attention.
- Prevents less important tasks from blocking urgent ones.
- Supports **real-time responsiveness** in embedded systems.

4.5. STM32 Interrupt Architecture



Contents

1. Program Query (Polling)
2. Interrupt-Driven Mode
3. Exception
4. Cortex-M3 Exception Model
5. STM32 Interrupt Architecture
   - NVIC
   - NVIC Priority
6. **External Interrupt (EXTI)**
7. Summary

4.6. External Interrupt (EXTI)

External Interrupt (EXTI)

- STM32 microcontrollers provide **dedicated EXTI (External Interrupt) lines**.
- These are special hardware connections that allow **external signals (from GPIO pins)** to trigger an interrupt.
- **Example**: A button press, motion sensor, or temperature alert.

4.6. External Interrupt (EXTI)

How it Works

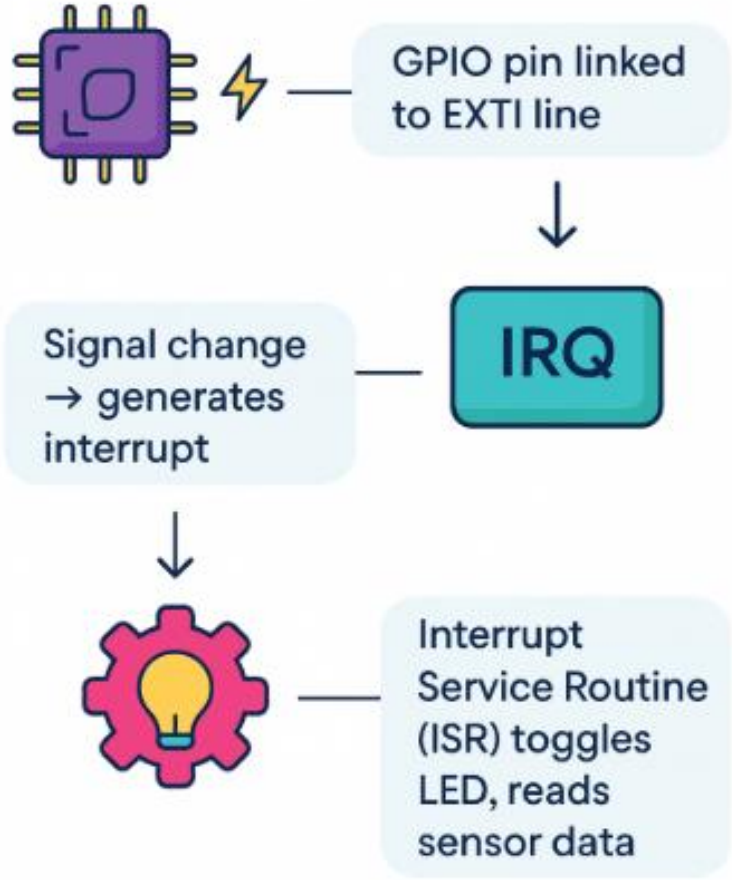
1.A **GPIO pin** is linked to an EXTI line.

2.When the input signal changes (depending on configuration),

it generates an **IRQ**.

3.CPU jumps to the **EXTI Interrupt Service Routine (ISR)**.

4.The ISR executes (e.g., toggles an LED, reads sensor data).



4.6. External Interrupt (EXTI)

Trigger Modes (Configurable)

- **Rising Edge:** Interrupt when signal changes from LOW → HIGH.
  - **Example**: Button press or sensor activation.
- **Falling Edge:** Interrupt when signal changes from HIGH → LOW.
  - **Example**: Button release.
- **Both Edges:** Interrupt for any change (press and release).

4.6. External Interrupt (EXTI)

Typical Applications

- **Switches/Buttons** → Detect user input.
- **Sensors** → Motion detector, IR sensor, etc.
- **Communication** → Detect start of a signal/data transfer.





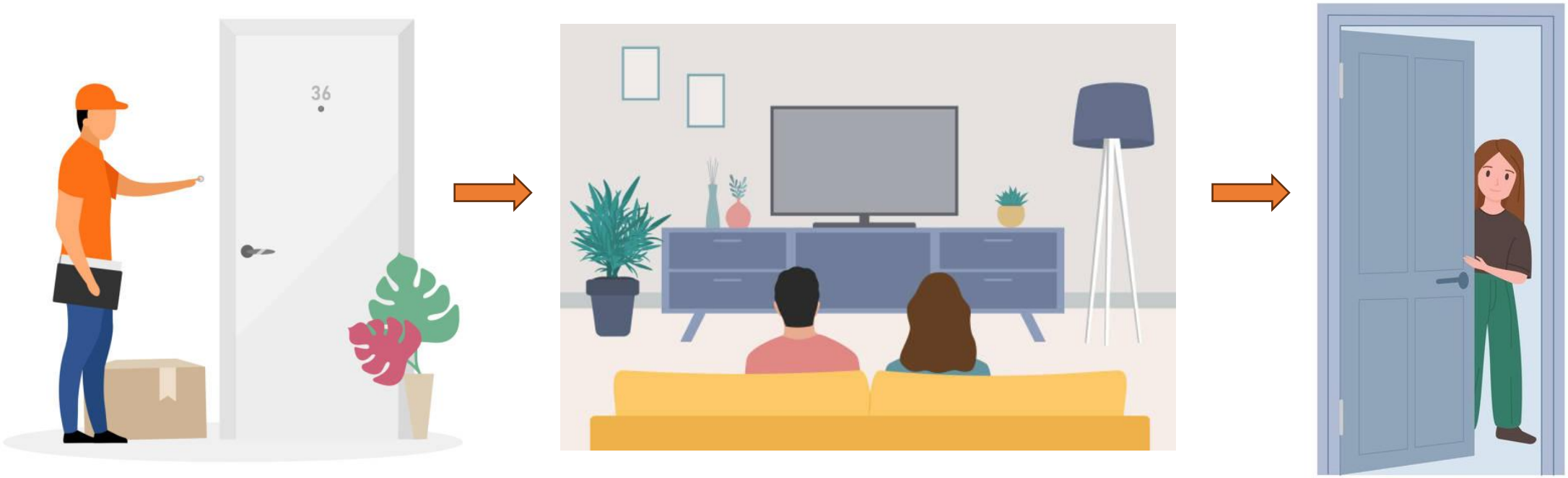




4.6. External Interrupt (EXTI)

Analogy: Imagine a **doorbell (EXTI pin)**:

- When someone presses the button (rising edge), the bell rings (interrupt).
- The CPU (you) stop your activity and go check the door.



Contents

1. Program Query (Polling)
2. Interrupt-Driven Mode
3. Exception
4. Cortex-M3 Exception Model
5. STM32 Interrupt Architecture
   - NVIC
   - NVIC Priority
6. External Interrupt (EXTI)
7. **Summary**

4.7. Summary

- **Polling vs Interrupts**
  - Polling = CPU keeps checking device status (wastes time).
  - Interrupts = Device notifies CPU only when needed (efficient).

- **Exceptions in Cortex-M3**
  - Events that change normal program flow.
  - Types: Reset, NMI, Faults, IRQs.

- **Reset Exception**
  - Occurs at power-on or reset button.
  - CPU starts from vector table → Reset Handler → main().

4.7. Summary

- **NMI (Non-Maskable Interrupt)**
  - Highest priority, cannot be disabled.
  - Used for urgent system errors.

- **Fault Exceptions**
  - Hard Fault = severe error.
  - Memory Fault = invalid memory access.
  - Bus Fault = data transfer error.
  - Usage Fault = invalid instruction/operation.

- **Interrupt Requests (IRQs)**
  - Generated by peripherals (GPIO, Timer, USART, ADC, etc.).

4.7. Summary

- **NVIC (Nested Vectored Interrupt Controller)**
  - Manages interrupts with priority levels.
  - Supports enable/disable and automatic context saving.

- **Priority Levels**
  - Preemption priority = decides if one IRQ can interrupt another.
  - Subpriority = decides order if priorities are equal.

- **External Interrupt (EXTI)**
  - Connected to GPIO pins.
  - Configurable as rising, falling, or both edges.
  - Commonly used for buttons and sensors.

